

Software Design Document

AI Shopping Assistant with Intelligent Product Recommendation System



Submitted by

Areeba Amir (S23BINFT1M01154)

Submitted to

Dr. Omer Riaz

Department of Information Technology

Faculty of Computing

The Islamia University of Bahawalpur

Summery

The AI Shopping Assistant is a web-based recommendation system designed to assist users in finding suitable products through a conversational interface. The system allows users to enter product-related queries in natural language, such as product type, budget preferences, and provides relevant product recommendations accordingly. The proposed system aims to simplify the product searching and decision-making process by providing intelligent suggestions based on user input.

The system is designed using the MVC (Model-View-Controller) architecture pattern and will be implemented using the Django framework. Basic Natural Language Processing (NLP) techniques such as tokenization and keyword extraction will be used through the spaCy library to process user queries and extract meaningful information like product category and price range. The extracted information will then be used by the backend to retrieve relevant products from the database and generate recommendations.

This document presents the overall software design of the proposed system, including application architecture, database schema, user interface design, controller functions, system workflow, and UML diagrams. The purpose of this document is to provide a clear technical design of the system before implementation.

Table of Contents

1. Application Architecture.....	5
1.1. Architecture Pattern.....	5
1.1.1. Model.....	5
1.1.2. View	6
1.1.3. Controller.....	6
1.2. Application Layers.....	6
1.2.1 Presentation Layer (Frontend)	7
1.2.2 Application Layer (Backend)	7
1.2.3 Data Layer	7
1.3 Controllers and Functions.....	7
1.3.1 Main Controllers	8
1.3.1.1. Query Controller	8
1.3.1.2. Product Controller	8
1.3.1.3. User Controller	8
1.3.1.4. Cart Controller.....	8
1.3.2 Functions.....	8
1.3.2.1. Query Processing.....	9
1.3.2.2. Product Recommendation	9
1.3.2.3. Product Display	9
1.3.2.4. Cart Management	9
1.3.2.5. User Authentication	9
1.4. External Libraries and APIs	9
1.4.1. Django Framework.....	10
1.4.2. spaCy Library	10
1.4.3. MySQL Database	10
1.4.4. HTML, CSS, and JavaScript	10
1.5. System Working Flow	10
1.6. Coding Conventions	11
1.7. UML Diagrams.....	11
1.7.1. Activity Diagram.....	12
1.7.2. Sequence Diagram.....	13
1.7.3. Class Diagram.....	13

2. Data Model Schema	14
2.1. Overview.....	14
2.2. Entities	15
2.2.1. Product Entries.....	15
2.2.2 User Entity	16
2.2.3 Cart Entity	16
2.2.4 Order Entity	17
2.3. Relationships	17
2.4. ER Diagram.....	17
3. User Interface	18
3.1. Design Philosophy.....	19
3.2. Screen and Pages.....	19
3.2.1. Home Page.....	19
3.2.2. Query Interface	20
3.2.3 Product Details Page.....	21
3.2.4 Cart Page	21
3.2.5 Login Page	21
3.2.6 Signup Page	22
3.2.7 Admin Panel	22
4. Conclusion.....	22

1. Application Architecture

The Application Architecture section describes the overall structural design of the proposed AI Shopping Assistant system. It explains how different system components interact with each other to perform frontend operations, backend processing, database management, and recommendation generation.

This section further explains the architectural pattern, application layers, controllers, backend functions, external libraries, system workflow, and coding standards used during the development of the system.

1.1. Architecture Pattern

The proposed AI Shopping Assistant system follows the Model-View-Controller (MVC) architectural pattern. The MVC architecture divides the system into three components: Model, View, and Controller, which improves system organization, maintainability, and scalability.

Within the system, the frontend interface manages user interaction and information presentation, while the backend handles application logic, query processing, recommendation generation, and database communication. The database layer stores product information, user records, and cart-related data.

The system is implemented using the Django framework. HTML, CSS, and JavaScript are used for frontend development, while MySQL is used as the database management system. Basic Natural Language Processing (NLP) techniques are integrated through the spaCy library to process user queries and support product recommendation generation.

1.1.1. Model

The Model component is responsible for managing and storing application data. It handles database operations and maintains communication with the database. The model component stores information related to products, users, cart items, and orders, while supporting data retrieval and update operations required by the backend system

The Product model stores product-related information such as product name, category, price, and description. Similarly, user-related models maintain authentication and cart management information. The Django ORM (Object Relational Mapping) is used to interact with the MySQL database efficiently.

1.1.2. View

The View component represents the user interface of the system. It is responsible for displaying information to users and collecting user input through web pages. The frontend interface of the proposed system is developed using HTML, CSS, and JavaScript.

The application includes multiple interface pages such as the homepage, recommendation interface, product details page, login page, signup page, and cart page. The homepage supports product browsing and query submission, while the recommendation interface displays products generated through backend processing. The View component ensures smooth and user-friendly interaction within the application.

1.1.3. Controller

The Controller component manages the application logic and acts as an intermediary between the View and Model components. It receives user requests from the frontend, processes the requests, communicates with the database, and returns appropriate responses to the user interface.

Within the system, the controller handles functionalities such as query processing, NLP-based keyword extraction, product recommendation generation, user authentication, and cart management. When a user enters a product-related query, the Controller processes the query using NLP techniques, extracts important keywords such as product category and price range, and retrieves matching products from the database. The processed results are then displayed to the user through the frontend interface.

1.2. Application Layers

The AI Shopping Assistant system follows a layered architecture approach to improve modularity, maintainability, and separation of responsibilities. The application is divided into three major

layers: Presentation Layer, Application Layer, and Data Layer. Each layer performs specific operations and communicates with other layers to support smooth system functionality.

1.2.1 Presentation Layer (Frontend)

The Presentation Layer manages user interaction and visual representation of the system. It allows users to browse products, submit queries, view recommendations, and manage shopping-related operations through the frontend interface developed using HTML, CSS, and JavaScript.

This layer focuses on improving user experience by providing a simple, responsive, and user-friendly interface similar to modern e-commerce platforms.

1.2.2 Application Layer (Backend)

The Application Layer handles core processing and business logic within the system. Implemented using Django, it processes user requests, manages authentication, performs NLP-based query processing through the spaCy library, and generates product recommendations based on extracted query information. This backend layer is also responsible for cart management, session handling, and communication with the MySQL database.

1.2.3 Data Layer

The Data Layer is responsible for storing and managing application data. MySQL is used to maintain product, user, shopping cart, and order-related information. The Django ORM (Object Relational Mapping) is used to establish communication between the backend application and the MySQL database. This approach simplifies database operations and improves data management within the system.

1.3 Controllers and Functions

The backend system contains multiple controllers and functions responsible for managing application operations. These components handle functionalities such as query processing, product management, recommendation generation, user authentication, and shopping cart operations.

Controllers manage communication between the frontend interface and database layer, while backend functions execute specific processing tasks required for recommendation generation and application management.

1.3.1 Main Controllers

The proposed system contains several controllers responsible for handling major application functionalities. These controllers are implemented within the Django backend framework.

1.3.1.1. Query Controller

The Query Controller handles user product-related queries entered through the user interface. It receives the query, performs NLP-based preprocessing operations, and forwards the processed information for recommendation generation.

1.3.1.2. Product Controller

The Product Controller manages product-related operations including retrieving product data, displaying product details, and filtering products based on extracted keywords.

1.3.1.3. User Controller

The User Controller handles user authentication functionalities including user registration, login, and session management and user access control.

1.3.1.4. Cart Controller

The Cart Controller manages shopping cart operations such as adding products to the cart, removing products, updating quantities, and displaying cart management.

1.3.2 Functions

The proposed system contains multiple backend functions responsible for executing application operations and recommendation logic.

1.3.2.1. Query Processing

This function processes the user query using Natural Language Processing techniques. It performs normalization, tokenization, stopword removal, lemmatization, and keyword extraction to identify important product-related information from the query.

1.3.2.2. Product Recommendation

This function retrieves relevant products from the database based on the extracted query information. The function filters products according to category, price range, and other relevant attributes before generating recommendations.

1.3.2.3. Product Display

Retrieves and displays detailed product information including product description, pricing, and image details on the product page.

1.3.2.4. Cart Management

This function manages shopping cart operations, including product addition, quantity updates, product removal, and cart information management.

1.3.2.5. User Authentication

This function manages user registration and login validation. It verifies user credentials and controls secure access to the system.

1.4. External Libraries and APIs

The proposed AI Shopping Assistant system utilizes several external libraries and technologies to support frontend development, backend processing, database management, and Natural Language Processing functionalities. These technologies improve development efficiency, maintain system organization, and support recommendation related operations within the application.

1.4.1. Django Framework

Django is used for backend development and application management within the proposed system. It handles user requests, URL routing, authentication, session management, and communication with the database. The framework also supports structured application development through the MVC-based architecture used in the system.

1.4.2. spaCy Library

The spaCy library is used to implement Natural Language Processing (NLP) functionalities within the system. The library performs operations such as text normalization, tokenization, stopword removal, lemmatization, and keyword extraction to process user product-related queries. These NLP techniques help the system identify important information of the query.

1.4.3. MySQL Database

MySQL is used as the database management system for storing and managing application data. The database stores information related to products, users, shopping carts, and order records. MySQL provides efficient data storage, retrieval, and management functionalities. The Django ORM is used to establish communication between the backend application and the MySQL database.

1.4.4. HTML, CSS, and JavaScript

HTML, CSS, and JavaScript are used for frontend interface development. These technologies are responsible for webpage structure, interface styling, responsive layout design, and interactive frontend behavior within the application.

1.5. System Working Flow

The proposed AI Shopping Assistant system follows a structured workflow to process user queries and generate product recommendations. Product information is initially stored in the MySQL database through the Django administration panel. When users access the system, they can browse products or submit product-related queries through the AI recommendation interface. The backend system processes user queries using NLP techniques implemented through the spaCy library to

extract meaningful product-related information. Based on the processed query, the recommendation module retrieves relevant products from the database and displays them through the frontend interface. Users can further view product details, manage shopping cart activities, and perform authentication operations such as login and signup. The overall workflow ensures effective communication between the frontend interface, backend processing module, NLP operations, and database management system.

1.6. Coding Conventions

The proposed AI Shopping Assistant system follows standard coding conventions and development practices to improve code readability, maintainability, and consistency throughout the application. The following coding practices are followed during system development,

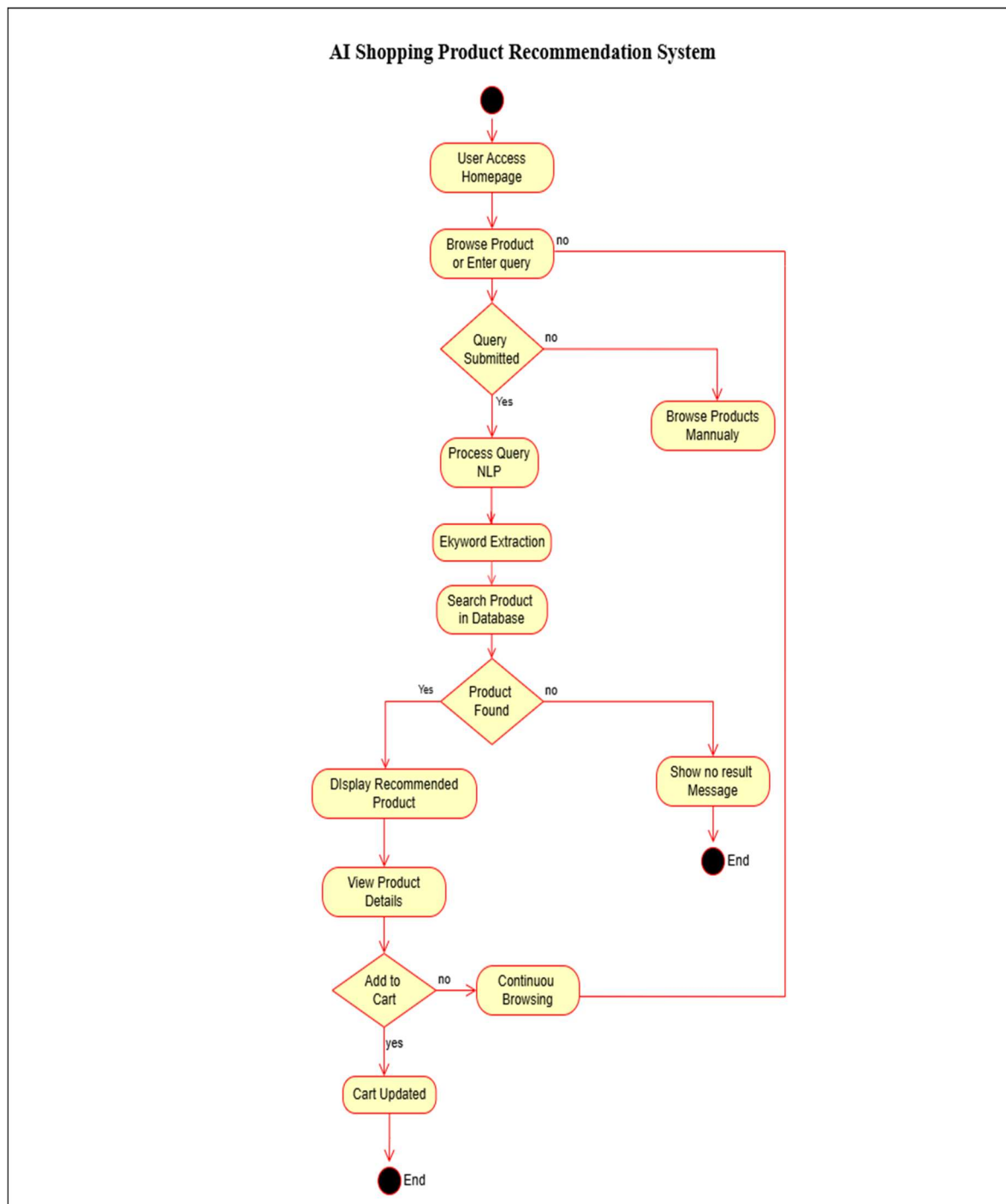
- Meaningful variable and function names are used for better code understanding.
- Proper indentation and formatting practices are maintained to ensure clean and readable code.
- HTML, CSS, JavaScript, and Python files are organized separately for better project management.
- Reusable functions and components are implemented to reduce code redundancy.
- Comments are added where necessary to explain important functionalities.
- The backend application follows Django project structure conventions and uses Django ORM for database communication.

1.7. UML Diagrams

This section contains the Unified Modeling Language (UML) diagrams of the proposed system to visually represent the structure, workflow, and interaction of the system. The following UML diagrams are included to represent the proposed system design.

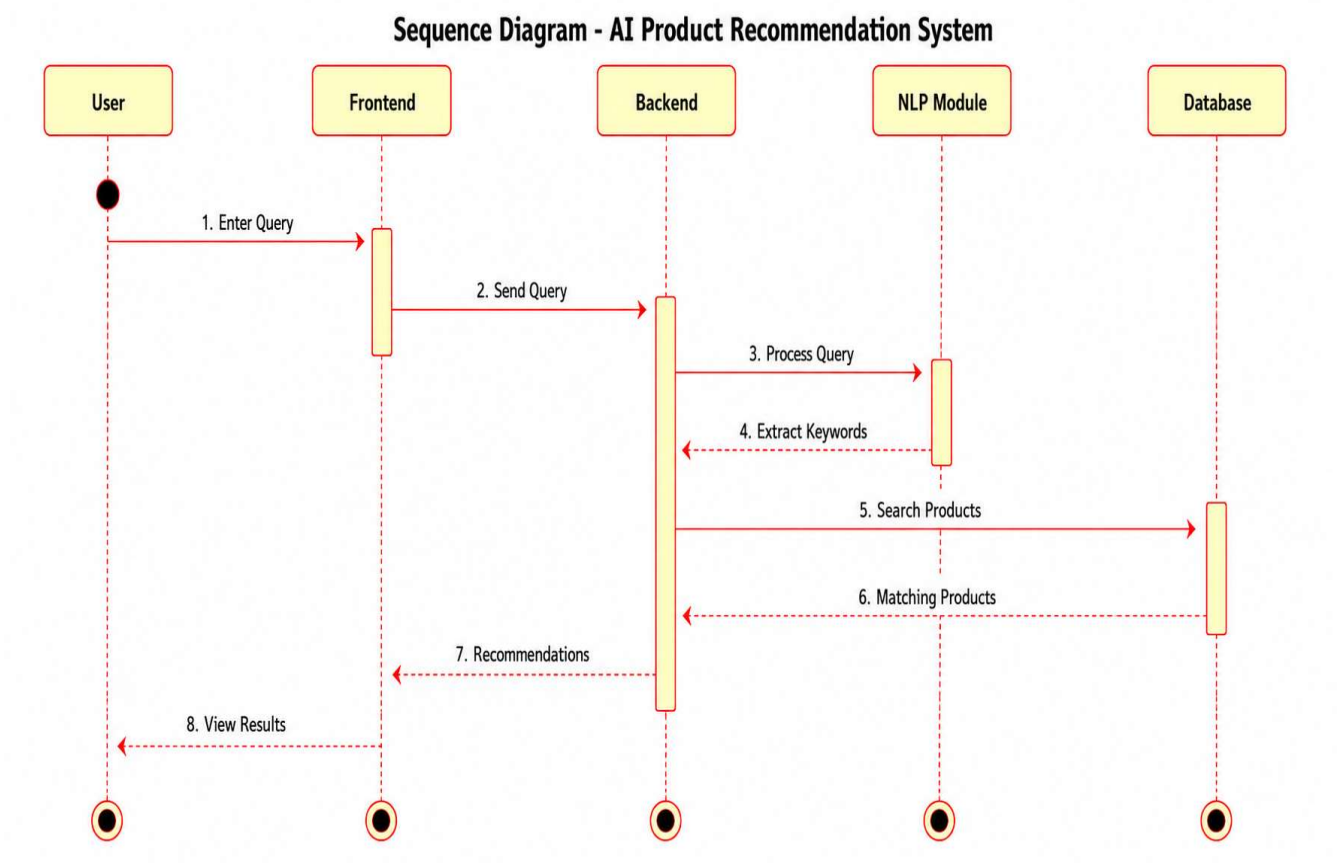
1.7.1. Activity Diagram

The activity diagram represents the overall workflow of the proposed AI Shopping Assistant system. It illustrates the sequence of activities involved in user interaction, query processing, product recommendation generation, and cart management within the system.



1.7.2. Sequence Diagram

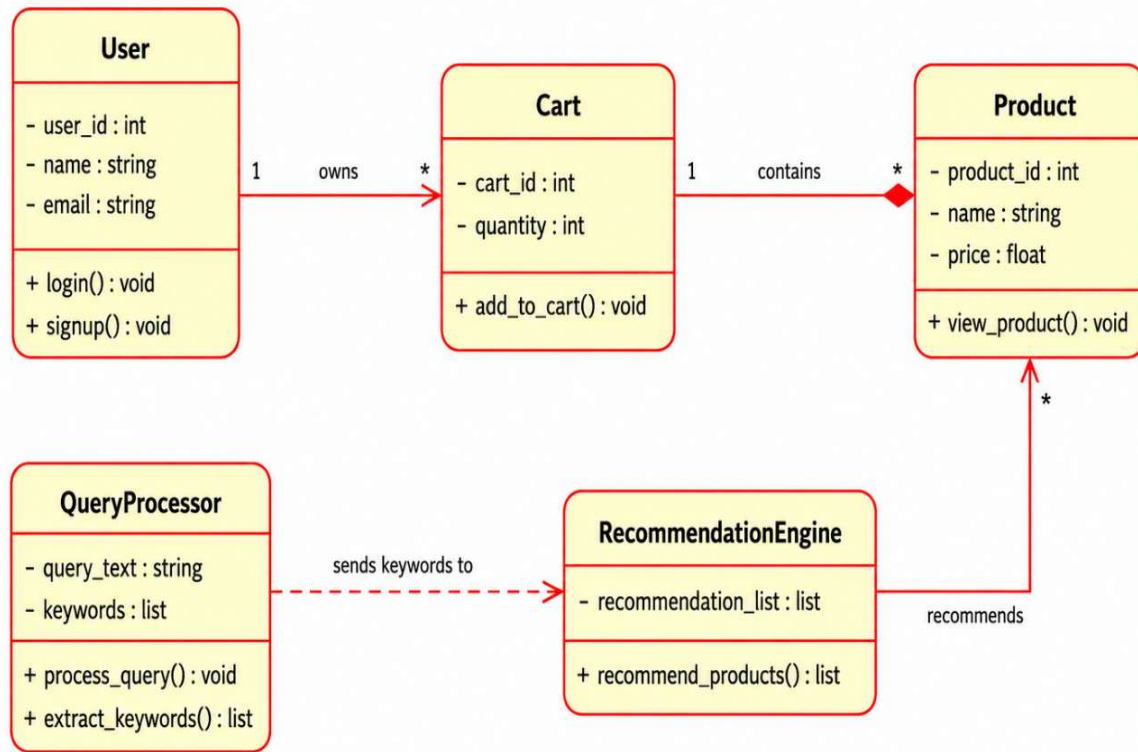
The sequence diagram illustrates the interaction between major system components during the product recommendation process. It shows the communication flow between the user interface, backend system, NLP processing module, and database



1.7.3. Class Diagram

The class diagram represents the structural design of the proposed system. It shows the major classes, their attributes, functions, and relationships involved in the application.

AI Product Recommendation System



2. Data Model Schema

Detail of database conceptual schema will be provided here. Each data model entity will be explained in details, its working properties and relationship with other entities. Authors will also provide ER diagram to explain the working entities.

2.1. Overview

The proposed system uses MySQL as the database management system for storing and managing application data. The database is responsible for maintaining product information, user records, shopping cart details, and order-related data required for system operations. The database design supports efficient storage, retrieval, and management of information within the application. The

backend system communicates with the MySQL database through Django ORM (Object Relational Mapping), which simplifies database operations and improves data consistency.

2.2. Entities

The proposed AI Shopping Assistant system contains multiple database entities used to organize and manage application data efficiently. Each entity represents a specific component of the system, such as products, users, shopping carts, and orders. These entities contain attributes that store required information for system operations and recommendation functionalities.

Primary Keys (PK) are used to uniquely identify records within each entity, while Foreign Keys (FK) are used to establish relationships between interconnected entities within the database.

2.2.1 Product Entity

The Product entity stores information related to products available within the system. This entity is responsible for maintaining product details required for product browsing, recommendation generation, and product display functionalities.

Attribute	Description
product_id	Unique identifier for each product
name	Product name
category	Product category
price	Product price
description	Product description
image	Product image information

2.2.2 User Entity

The User entity stores information related to registered users of the system. It maintains user authentication and account-related information required for secure system access and shopping activities.

Attribute	Description
user_id	Unique identifier for each user
username	Username of the user
email	User email address
password	User account password
contact_number	User contact information

2.2.3 Cart Entity

The Cart entity stores shopping cart information associated with users. It maintains records of products selected by users for shopping-related operations.

Attribute	Description
cart_id	Unique identifier for cart
user_id	Associated user identifier
product_id	Associated product identifier
quantity	Quantity of selected products
total_price	Total cart price

2.2.4 Order Entity

The Order entity stores information related to completed shopping operations and user purchase records.

Attribute	Description
order_id	Unique identifier for each order
user_id	Associated user identifier
total_amount	Total order amount
order_date	Date of order placement

2.3. Relationships

The database entities within the proposed AI Shopping Assistant system are interconnected to support product browsing, shopping cart management, and order processing operations. These relationships improve data organization and ensure efficient communication between different components of the application.

The User entity is associated with the Cart entity, allowing users to store selected products within the shopping cart. The Cart entity is connected with the Product entity through product identifiers used to maintain selected product information.

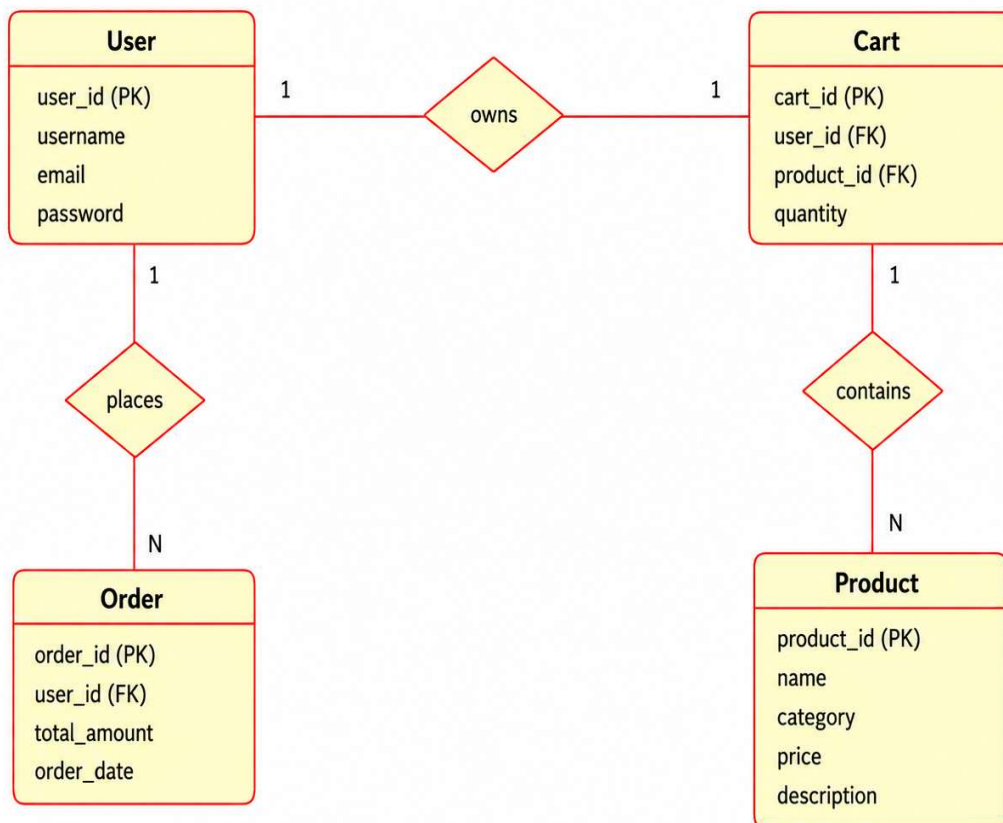
Similarly, the User entity is linked with the Order entity to store purchase records and shopping history associated with registered users. These relationships ensure consistency and efficient data management within the system database.

2.4. ER Diagram

The Entity Relationship (ER) diagram represents the database structure of the proposed AI Shopping Assistant system. It illustrates the major database entities, their attributes, and

relationships used for managing products, users, shopping carts, and order-related operations within the application.

AI Product Recommendation System



3. User Interface

The User Interface of the proposed AI Shopping Assistant system is designed to provide an organized and interactive shopping environment for users. The interface supports product browsing, query-based recommendation generation, and shopping-related operations through a

structured e-commerce inspired layout. The frontend design focuses on accessibility, smooth navigation, and consistent interaction throughout the system.

3.1. Design Philosophy

The proposed system follows a modern and minimal ecommerce-inspired design philosophy to maintain a clean and organized interface structure. The interface is designed to simplify product browsing and support NLP-based product recommendation interaction through a straightforward and responsive layout.

The frontend interface uses a structured arrangement of navigation sections, product cards, search functionality, and recommendation interfaces similar to contemporary ecommerce platforms. The design emphasizes readability, visual consistency, and responsive behavior to support interaction across desktop and mobile devices.

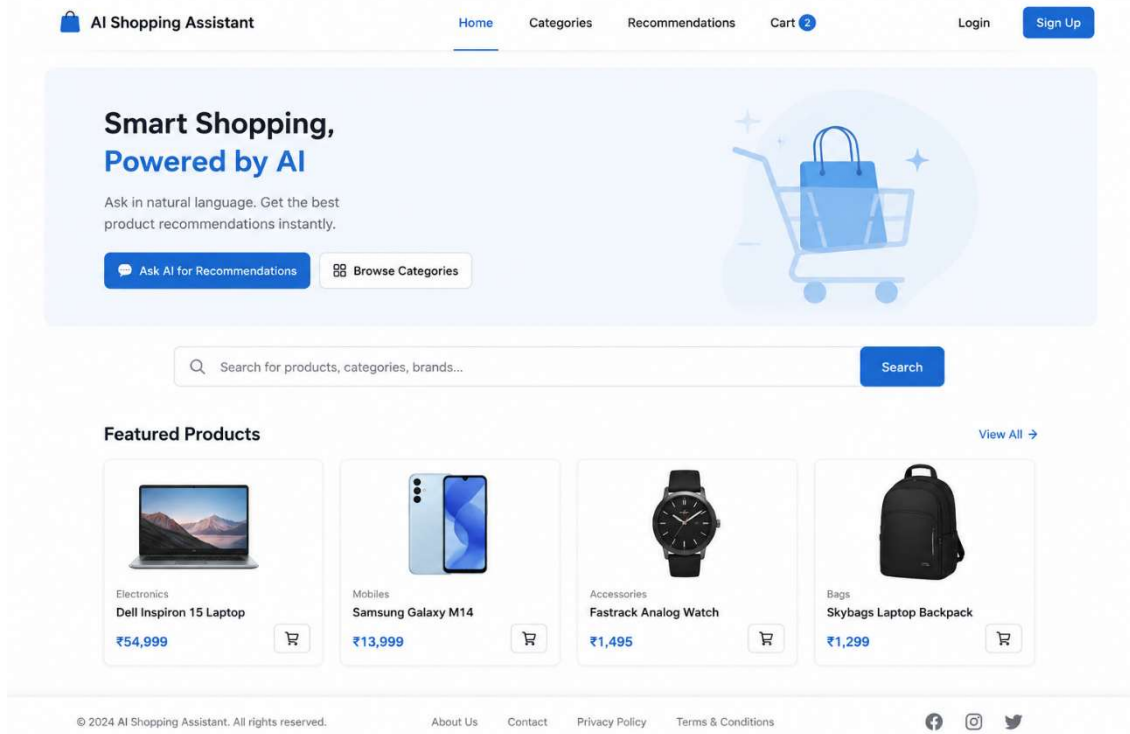
The user interface is developed using HTML, CSS, and JavaScript. Responsive layout techniques are used to maintain consistent navigation and interface behavior across different screen sizes.

3.2. Screen and Pages

The proposed AI Shopping Assistant system consists of multiple interface pages designed to support browsing, recommendation generation, user authentication, and shopping management operations. Each page performs a specific role within the application while maintaining consistency with the overall frontend structure.

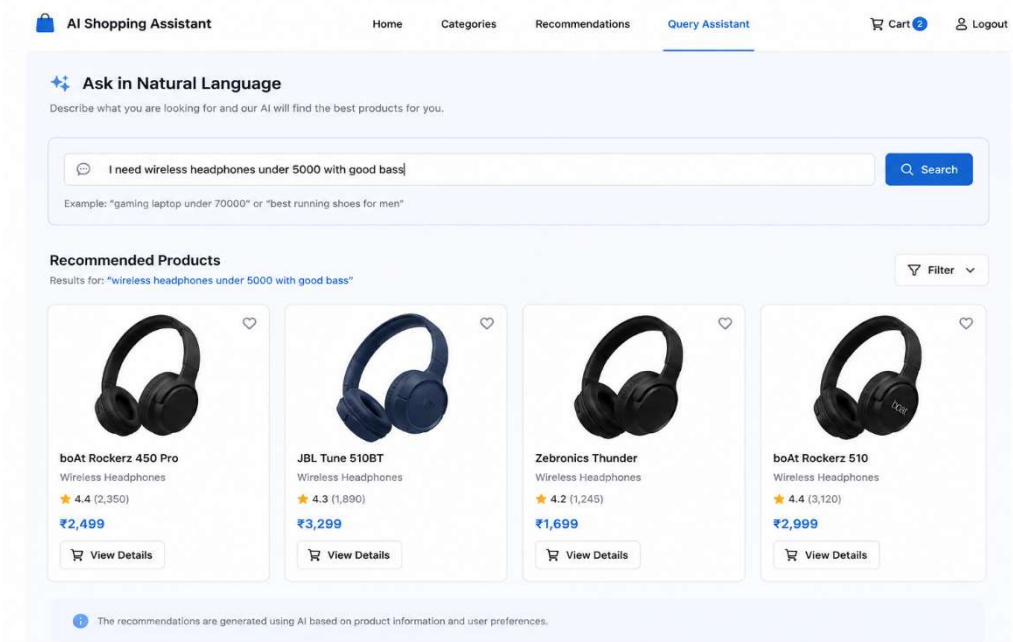
3.2.1. Homepage

The Homepage serves as the main interface of the system. It provides product browsing functionality, navigation options, featured products, and access to the AI query interface. The page contains a navigation bar, search section, product cards, and recommendation access options for users.



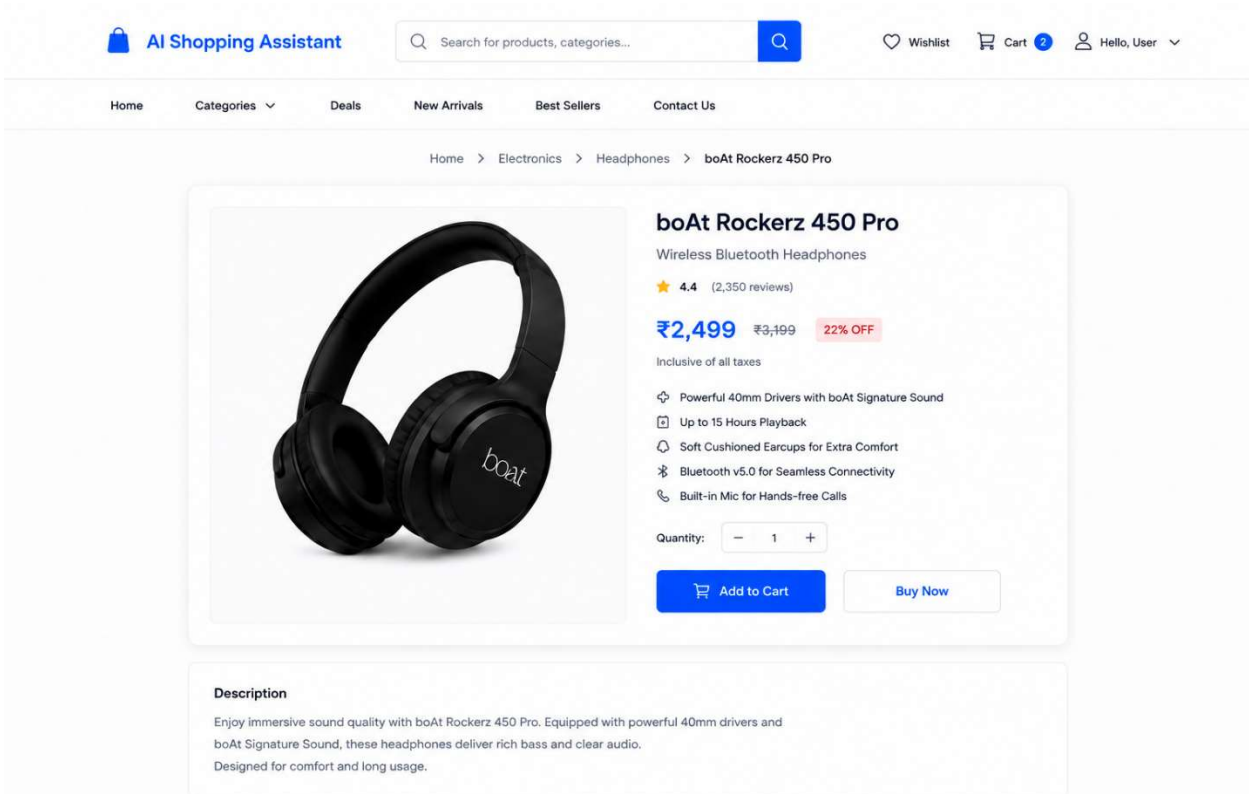
3.2.2. Query Interface

The Query Interface allows users to submit product-related queries in natural language form. The backend system processes the query using NLP techniques and displays relevant product recommendations based on extracted keywords and user preferences.



3.2.3 Product Details Page

The Product Details Page displays complete information related to a selected product including product name, category, price, description, and product image. The page also allows users to add products to the shopping cart.



3.2.4 Cart Page

The Cart Page displays products selected by users for shopping operations. Users can update product quantities, remove products from the cart, and review the total shopping amount before checkout operations.

3.2.5 Login Page

The Login Page allows registered users to access the system securely through email and password authentication. The backend system verifies user credentials before granting access to shopping functionalities.

3.2.6 Signup Page

The Signup Page allows new users to create accounts by providing required registration information. Input validation mechanisms are used to maintain proper data entry and user management consistency.

3.2.7 Admin Panel

The Admin Panel allows administrators to manage product-related information within the system. Administrators can add, update, remove, and organize product records stored in the database.

4. Conclusion

The proposed AI Shopping Assistant system is designed to support product browsing and NLP-based product recommendation within an ecommerce environment. The system integrates frontend interaction, backend processing, database management, and natural language query handling to provide efficient shopping-related functionalities.

The overall system architecture, database design, UML diagrams, and user interface collectively represent the technical structure and workflow of the proposed application.